

Design Note — Learning from Click-Logs on EB-NeRD and MIND

CS4.406 Information Retrieval & Extraction — Assignment 2

September 18, 2026

1 What We Built

Assignment 1 built lexical (BM25) and semantic (embedding) candidate generation for both datasets. Assignment 2 adds the behavioural layer: engineered click-history features, a two-stage retrieve-then-rank pipeline, and — the graded gap A1 left open — a literal reproduction of each dataset’s official NRMS baseline, improved by one principled change and tested with a paired bootstrap.

1.1 Click-history and session features (Q1)

EB-NeRD already had 65 engineered features (Candidate K, ADR-013): long/short-term category/topic/embedding affinity, freshness (`article_age_h`, the single top feature by gain), and — added this assignment for Q1 — session features (`session_position`, `session_start_gap_h`, three `context_*` match features). MIND had none of this: only frozen-embedding similarity and symbolic category/entity overlap (Candidates A–J).

Candidate L (A2 Q1, this assignment) adds five engineered features for MIND: click count, category match, subcategory match (the last two already existed unweighted in `features.py`), and two new **recency-weighted** affinity features using the same geometric decay (decay= 0.9, most-recent-last) already established for MIND’s embedding-query recency experiment (Candidate C). Scored by a cheap CPU-only logistic-regression combiner — deliberately not a new NRMS run, so it does not contend with the treatment training job below.

Result: a real, CI-clear loss. AUC 0.6176 [0.6153, 0.6197] vs. the deployed embedding baseline’s 0.6340 — paired -0.0164 [-0.0191 , -0.0139], and every metric (MRR, nDCG@5, nDCG@10) loses the same way (ADR-016). Reported plainly, not explained away: no hyperparameter search was run, and the CI half-width (~ 0.001 – 0.002) is far smaller than the 0.016 gap, so tuning would not close it. Click-history statistics alone — recency-weighted or not — do not carry enough per-candidate signal to beat frozen embedding similarity on MIND, consistent with Candidate D’s earlier standalone loss (0.613) and with Candidate F’s finding that even a 7-feature combiner including embeddings only barely avoided losing overall. Not adopted; MIND’s real win remains Candidate J and, per this assignment, the reproduced-then- improved official NRMS (§2.2).

Stated as a dataset limitation, not a gap to force-fill: MIND has *no* freshness feature and *no* session feature. Confirmed by direct measurement against the processed data, not assumed — `article_published_time` is null for all 72,023 MINDlarge articles, and `session_id/dwell_time/scroll_percentage/is_front_page` are null for all 14,085,557 impression rows. `src/datasets/mind.py` hardcodes these columns to `None` at parse time because MIND’s source TSVs never carry this information at all — it is not a sampling artifact and cannot be engineered around.

A modeling fact worth stating plainly: click count is identical for every candidate within one impression (it depends only on the user). A linear combiner’s contribution to a per-impression ranking metric (AUC/MRR/nDCG@K — all rank-order-only) is provably invariant to an additive per-impression constant. Rather than asserting this from theory alone, the experiment script includes an ablation that zeroes the feature and re-scores every dev impression: the maximum absolute AUC change was **exactly** 0.00e+00 — not approximately zero, bit-for-bit zero, matching the proof precisely. Click count is still reported, per Q1’s explicit ask, with this caveat attached

rather than hidden; its fitted coefficient (0.064) is nonzero because it can still reduce the pointwise training loss (which differs across users) even though it cannot move any impression’s internal ranking.

Leakage. MIND provides no per-click timestamps, so the EB-NeRD-style temporal-boundary check cannot be built for MIND. The structurally equivalent guard the dataset actually supports is reused instead: since a user’s history is one fixed snapshot for the whole split, every history-derived feature must trace back to *exactly one* value regardless of which of that user’s real impressions produced it. A new integration test exercises this against real MINDsmall data (not a synthetic call-the-function-twice check, which would prove only determinism) and confirms candidate-derived components legitimately vary while history-derived ones do not.

1.2 Two-stage retrieve-then-rank, and an honest retrieval ceiling (Q2)

Both leaderboard submissions rank each impression’s own shown candidate list — what the competitions actually score. A2 additionally asks for top- K retrieval from A1’s own generator, $K \sim 100$ –200, re-ranked, with before/after metrics reported. This is built as a literal, runnable harness (`scripts/a2_q2_retrieve_rerank_eval.py`), not left as a citation: A1’s retriever pulls the top- K candidates from the *whole corpus* per sampled impression using the user’s real history as the query, and the trained OfficialNRMS re-ranks that retrieved list. The retrieval ceiling is measured fresh on the same sample rather than re-cited from an earlier benchmark run, via the same bootstrap machinery as every other metric in this project. A1’s independently measured recall@200 (2.62–2.78% MIND, 1.21–2.77% EB-NeRD) means at least $\sim 97\%$ of clicked articles never enter a top-200 list from either retriever — no re-ranker can recover a click the candidate generator never surfaced, and the harness’s own freshly-measured hit rates confirm this order of magnitude directly (3.70%, CI 3.05–4.37%, BM25 on a 3,000-impression MINDsmall-dev sample).

The full-scale run: a large, CI-clear win for re-ranking. Using the real treatment checkpoint (10 epochs, MINDlarge, dev AUC 0.6868 — matching §2.2’s Q3 result to four decimal places) on 8,000 sampled MINDlarge-dev impressions: hit rate 3.21% (CI 2.83–3.61%, consistent with the ceiling above), and among the impressions where the retrieved 200 actually contain the real click, NRMS re-ranking lifts AUC from 0.5428 (raw BM25 order, barely above random) to **0.7224** — paired +0.1796 [+0.1375, +0.2213], with MRR/nDCG@5/nDCG@10 all CI-clear wins in the same direction (`results/a2_q2/README.md`). An earlier interim check, run while this session’s environment could not yet reach the GPU cluster holding the trained checkpoint, used a deliberately-flagged 1-epoch/20,000-impression local checkpoint and found no significant effect (paired AUC –0.0464, CI spanning zero) — exactly the undertrained-checkpoint artifact that check was labeled as, now superseded by the real result. The honest overall answer: re-ranking helps substantially *when* retrieval succeeds, but cannot recover the $\sim 97\%$ of clicks retrieval never surfaces at all — both halves of Q2’s answer, not one standing in for the other.

Only NRMS (both datasets) re-ranks in that evaluation. Candidate K’s GBDT is excluded by construction: its features are *impression-conditional* (`position_in_view`, the five `*_rank_in_imp` features, `n_candidates`, three `context_*` features), defined relative to the other candidates the publisher actually showed. A retrieved-but-never-shown candidate has no position in that list and no in-list rank to compare against — any value would be invented and would silently drive the ranking. A reduced “retrieval-safe” GBDT was considered and rejected: it would answer a different model’s ceiling, not Candidate K’s, at the cost of a second model to maintain.

2 Baseline Reproduced, Then Beaten (Q3) — A/B Framing

Control = the official baseline, reproduced. **Treatment** = control plus exactly one principled change. **Test** = paired bootstrap 95% CI of (treatment – control) over the identical impressions,

resampling users — the same statistic ADR-010’s Candidate G introduced. **Guardrails** that must not regress even if the primary metric wins: diversity@10, novelty@10, coverage@10, and serving latency (p99, §3).

Reproduction method. The brief names the official code directly: ebnerd-benchmark’s NRMS for EB-NeRD, `microsoft/recommenders`’ NRMS for MIND. We attempted this (Option A) on Ada within a one-session timebox; it did not train — every reachable machine’s PyPI throughput (0.09–0.37 MB/s) made the ~3 GB TensorFlow+CUDA install multi-hour, and one job held a GPU idle ~4h before we diagnosed it. Per the pre-agreed timebox rule, we fell back to **Option B**: a layer-for-layer PyTorch port of the official Keras graph (bias-free Q/K/V attention at the official 20×20 width, no output projection, no padding mask, the exact `AttLayer2` pooling formula, official hyperparameters read from the shipped config files rather than retyped) plus the official loaders’ exact input semantics (most-recent-50 history, MIND’s zero-pad negative sampling, EB-NeRD’s with-replacement sampling). This must be read as *a faithful reimplementaion*, never as “the official code” — and it is reported as such throughout, with the revisit condition (a cheap TF environment) recorded rather than silently dropped.

2.1 EB-NeRD — complete

	AUC	95% CI	MRR	nDCG@5	nDCG@10	Div.@10	Nov.@10
Control (reproduced NRMS)	0.5613	0.5597–0.5629	0.3512	0.3904	0.4680	0.7894	17.208
Treatment (+freshness)	0.5677	0.5659–0.5696	0.3555	0.3957	0.4725	0.7900	17.236

Paired treatment – control: AUC +0.0064 [+0.0053, +0.0075]; MRR +0.0043; nDCG@5 +0.0053; nDCG@10 +0.0044 — all CI-clear. **No guardrail regressed:** diversity +0.0006 and novelty +0.0280, both CI-clear improvements; coverage fell 0.2021 → 0.1987 (–1.7% relative), reported as a point difference only (ADR-007: a set-union statistic is mechanically biased under bootstrap resampling), flagged rather than hidden.

Two things the headline must not obscure. **The gain is small and merely *tightly* estimated** — the narrow CI reflects $n = 244,647$ impressions, not a large effect. **The improved baseline (0.5677) is still far below Candidate K (0.7588)** on the identical split; A1’s GBDT remains the strongest EB-NeRD system this project has, and beating the official NRMS baseline does not change that.

Head/tail slicing (Q5) reverses the headline. An impression’s clicked article is *head* if it sits in the top 20% of train-split click counts, else *tail* (train-only, so no dev-click leakage; never-clicked-in-train articles fall in tail by construction). The paired gain is entirely a tail effect:

Slice	n	Paired Δ AUC (95% CI)	Verdict
Head (top 20% by train clicks)	9,541	–0.0217 [–0.0258, –0.0178]	CI-clear loss
Tail	235,106	+0.0076 [+0.0064, +0.0087]	CI-clear gain

The +0.0064 aggregate is a tail gain diluted by a real head regression on 3.9% of impressions — exactly the failure mode slicing exists to catch. Mechanistically plausible: a freshness-weighted ranker should help least where an article is popular enough to be clicked regardless of age.

Anti-gaming (Q9): pricing the serving-time-unavailable features. `context_read_time/context_scroll_percentage` were flagged in ADR-013 as EB-NeRD’s weakest serving-time claim (a page’s total read time is only known once the user leaves it). A fourth GBDT arm withholding both gives 0.7570 vs. 0.7581 with them — paired +0.0011 [+0.0008, +0.0015], CI-clear but negligible against the +0.2140 margin over `embed_sim`. For contrast, withholding *position* features *improves* AUC by 0.0007. Neither flagged group is load-bearing; freshness is (`article_age_h` remains top by gain). `K_rank` and `K_rank_nopos` reproduced bit-identically across the three- and four-arm runs,

incidental evidence of a deterministic pipeline.

2.2 MIND — complete

	AUC	95% CI	MRR	nDCG@5	nDCG@10
Control (reproduced NRMS, 10 ep.)	0.6831	0.6821–0.6839	0.3803	0.3620	0.4279
Candidate J (A1’s best, same split)	0.6579	—	0.3581	0.3411	0.4064
Published NRMS (paper, <i>test</i> split)	0.6776	—	0.3305 [†]	0.3594	0.4163

[†] Not directly comparable: the official evaluator sums 1/rank over *all* clicked items; ours is first-hit-only (established in A1’s Codabench converter validation).

The reproduced control **beats Candidate J by +0.0252 AUC** on the identical MINDlarge-dev split (marginal, not paired — J’s per-impression scores were never saved) and lands close to the published test-split figure, which is the evidence Q3 needs that the reproduction is trustworthy. Head/tail and warm/cold slices are already computed for the control: head 0.7411 vs. tail 0.6474; warm 0.6932 vs. cold 0.6209 — a 0.0723 gap, marginally *wider* than A1’s embedding-baseline gap (0.0682), so the reproduced baseline lifts both cohorts ~ 0.05 without closing cold-start, echoing ADR-008’s conclusion for embeddings.

Treatment: title+abstract input (title 30 tokens + abstract 50, widths from the official `nrms.yaml/naml.yaml`), ablated against the title-only control. *The original treatment spec — GloVe vs. random init — was reconsidered before running it:* the official NRMS is already GloVe-initialised, so that comparison would have measured the control against a deliberately degraded control, not an improvement. Training took 34.06h (10 epochs, no errors), comfortably inside the 72h wall limit.

	AUC	95% CI	MRR	nDCG@5	nDCG@10	Div.@10	Nov.@10
Control (reproduced NRMS)	0.6831	0.6821–0.6839	0.3803	0.3620	0.4279	0.8343	17.623
Treatment (+title/abstract)	0.6868	0.6859–0.6877	0.3829	0.3691	0.4314	0.8283	17.643

Paired treatment – control: AUC +0.0037 [+0.0031, +0.0044]; MRR +0.0026; nDCG@5 +0.0071; nDCG@10 +0.0034 — all CI-clear wins. **A guardrail genuinely regressed:** diversity –0.0060 [–0.0062, –0.0057], CI-clear, while novelty improved +0.0197 (CI-clear) and coverage was flat (–0.0001, point only). **This is the first guardrail regression in this project’s entire history** — every earlier win, including EB-NeRD’s freshness treatment above, improved every CI-bearing guardrail. Per the professor’s explicit framing, a guardrail regression is not something a primary-metric win is allowed to paper over: it is reported as a real cost of the change, not smoothed into the accuracy headline. Plausible mechanism: 49 additional abstract tokens likely sharpen the model’s confidence toward specific topical matches at some expense to the categorical variety of what it ranks highly.

Head/tail slicing independently reproduces EB-NeRD’s exact directional pattern — a different dataset, a different treatment, a different mechanism, the same qualitative result: head –0.0063 [–0.0071, –0.0053] (CI-clear loss, $n = 143,164$) vs. tail +0.0098 [+0.0090, +0.0108] (CI-clear gain, $n = 233,307$). Both treatments help most where the content signal is weakest relative to prior popularity and can mildly hurt where an article is already identifiable from its title alone — independent replication, not designed in advance. **The gain is entirely a warm-user effect:** warm +0.0043 [+0.0036, +0.0050] CI-clear, cold +0.0002 [–0.0020, +0.0022] *not significant* — sensible, since richer candidate text cannot improve a user representation built from zero history. The reproduced-then-improved pipeline still does not close cold-start, consistent with every earlier finding in this project.

Shipped as the new MIND leaderboard submission — a real, confirmed win. The –0.0060 diversity regression is real (above) but is relative to the reproduced control, not to what

was actually deployed: measured against Candidate J’s real deployed diversity@10 (0.7840, from J’s own submitted predictions), the treatment is a diversity *improvement* of +0.0443 — an order of magnitude larger than the regression it costs relative to the lab-only control (ADR-015’s 2026-09-14 addendum). Real MINDlarge-test predictions were generated from the actual trained checkpoint (2,370,727 lines, independently verified: exact line-count match, every line a valid permutation, zero duplicates or malformed rows) and packaged as `prediction.zip`. A first upload (submission 930353) scored an anomalous 0.5589 — a real pipeline bug (a user-id key mismatch silently scored every impression with empty history), root-caused via an independent re-scoring spot check and fixed at the source (ADR-015’s 2026-09-18 addendum), not a genuine generalization failure. The corrected file, regenerated from the same unmodified checkpoint, was uploaded as submission 930992: **real Codabench score 0.6766** — inside this project’s normal dev-to-test compression range (0.6868 → 0.6766) and a real +0.0304 over the previous official submission (901961, 0.6462). 930992 is now the selected official MIND submission.

3 Serving and Scale Analysis (Q4)

Everything below is a measured *single-request* p99, not the batched, amortised figure this project’s stage-level profiling (ADR-014) originally reported — the gap between the two is itself a finding (below).

Instance	Index/store	p99	SLA 100ms	\$/1k (1 worker)
MIND / CPU (c6i.2xlarge)	164.0 MB	94.67 ms	PASS, 1.1×	\$0.0031
MIND / GPU (g4dn.xlarge)	164.0 MB	33.15 ms	PASS, 3.0×	\$0.0007
EB-NeRD (<code>nopos</code>) / CPU	190.5 MB	22.74 ms	PASS, 4.4×	\$0.0009

Both MIND rows are genuinely measured on their own hardware, not one profile priced onto both instance types — an early version of the cost script did exactly that (excluded rather than reported the invalid row), and a second, mirrored version of the same bug (this time on the CPU row, once the GPU profile became the newest file) was caught and fixed with a regression test before these numbers were trusted.

Batched vs. single-request differs by mechanism, and by how much room there is to amortise. EB-NeRD’s $\sim 30\times$ gap (0.33 ms/impression amortised vs. ~ 10 ms for one cold request) comes from per-call vectorisation overhead paid once per 5,000-impression chunk instead of once per impression. Quoting the batched figure as request latency would have understated EB-NeRD’s real serving cost by a factor of thirty. MIND’s batched sweep instead hoists per-*user* work (tokenize, BM25/embedding scoring) out of the impression loop — but the real MIND population averages only 1.47 impressions/user (ADR-014), so there is little to amortise, and the batched figure (40.46 ms/impression) and the cold single-request mean (33.29 ms) sit within $\sim 1.2\times$ of each other, in the *opposite* direction from EB-NeRD’s gap. Reported as measured, not smoothed into a false parallel between the two datasets.

On CPU, MIND’s p99 headroom is thin, and its observed max (126 ms) already breaches the 100 ms target — NRMS is 78% of a request’s cost there. **On GPU, the same target has real headroom (3.0×**: NRMS drops to 36% of the request, matching this project’s own GPU-vs-CPU finding elsewhere in ADR-014 (NRMS 8.98× faster on a 2080 Ti). EB-NeRD is comfortable on CPU alone (4.4×) and dominated by feature-frame construction, not the booster, so a GPU figure was not pursued for it.

The **1-worker** \$/1k figures above are the conservative end of a range: an *optimistic* column (not shown) assuming perfect linear scaling across an instance’s vCPUs would give roughly 8× lower figures, but this project’s own profiling contradicts that assumption directly (a measured 2.5× slowdown from a single concurrent job on one 8GB machine, ADR-014) — real per-instance throughput

is sublinear, so the 1-worker number is the more honest one to lead with. Prices are AWS on-demand list, us-east-1, captured 2026-09-12.

4 Where the System Breaks at $10\times$

MIND breaks on CPU latency, not memory — and GPU serving is a measured fix, not a hoped-for one. The index (164 MB) scales with articles/vocabulary, not users — MINDlarge-test’s 120,959 articles project to ~ 250 MB, still trivial. On CPU, p99 headroom is only $1.1\times$ with NRMS at 78% of a request; **on GPU, the identical request-level benchmark measures $3.0\times$ headroom** (§3), not a projection from a training-time speedup. A second, complementary fix needs no new hardware at all: this project’s own NRMS candidate-vector cache measures $7.02\times$ on CPU with no accuracy cost (ADR-014). At $10\times$ load, GPU serving buys real room but not unlimited room — $3.0\times$ headroom absorbs roughly a $3\times$ traffic increase before the SLA is threatened again, so $10\times$ load still needs horizontal scaling (more GPU workers) on top of the device change, not instead of it.

EB-NeRD breaks on memory, specifically user profiles. They cost 5.35 KB/user (82.1 MB for 15,342 users); the article table (108.2 MB) scales with articles instead. At `ebnerd_large`’s real 791,582 users, the same per-user rate projects to ~ 4.2 GB of profiles alone — from a 190 MB feature store today to something that no longer sits comfortably beside a model on a small serving instance. Latency is not the constraint here ($4.4\times$ headroom).

The treatment itself has a real $10\times$ -adjacent cost story. MIND’s title+abstract treatment costs $2.98\times$ the control per training step — measured, not assumed, from the first 45,000 of 105,740 steps before committing the full ~ 37 h job. The same accounting applies to any future feature addition: verify the marginal training cost on a small slice before paying for the whole run, which is exactly how the 37h estimate was validated here rather than discovered as an overrun.

5 Limitations and Open Items

- Q9’s serving-time-unavailable ablation and Candidate K’s 65-feature retrain are measured on `ebnerd_small`, not the `ebnerd_large` split behind the real 0.7542 Codabench score — that bundle is unreachable at Ada’s measured ~ 18 KB/s S3 rate with the cluster home directory’s remaining ~ 4.9 GB.
- Reproducibility is bitwise on CPU (verified: the reproduced-NRMS smoke tests and the EB-NeRD control’s independent re-run matched to every printed digit) but only stable to ~ 8 decimals run-to-run on GPU (ordinary CUDA reduction non-determinism, confirmed not to be a code defect) — stated because this project’s principle is “same code + same data = same results,” and the GPU qualification is real, not glossed over.